

BITÁCORA DE DESARROLLO

Fisser HandBeam 10D

Interfaz Gráfica con EEZ Studio + LVGL

Herramientas utilizadas:

EEZ Studio | LVGL 9.x | EEZ Flow | Python 3.12 | pngquant

Proyecto: UI embebida en hardware con pantalla táctil capacitiva

1. Introducción y Contexto del Proyecto

Este documento es la bitácora técnica completa del proceso de diseño e implementación de la interfaz gráfica del dispositivo médico Fisser HandBeam 10D, un equipo de terapia láser de baja intensidad con 10 LEDs (4 rojos, 3 violetas, 3 infrarrojos).

La interfaz fue construida íntegramente con EEZ Studio usando LVGL 9.x como backend gráfico. El desarrollo incluyó múltiples pantallas, lógica de negocio mediante EEZ Flow, manejo de variables globales, y comunicación bidireccional entre la UI y el hardware subyacente.

Pantallas implementadas

- SplashScreen — Pantalla de arranque con animación de carga
- login_SCREEN — Pantalla de acceso con PIN de 4 dígitos y teclado numérico
- MENU — Pantalla principal de tratamiento con checkboxes de LEDs, timer y dosis
- SETTINGS — Pantalla de configuración general
- CAMBIO_PASSWORD — Pantalla para cambiar el PIN de acceso

2. Pantalla de Login (login_SCREEN)

2.1 Objetivo

Crear una pantalla de autenticación con teclado numérico (ButtonMatrix) que permita ingresar un PIN de 4 dígitos. El botón del ojo debe mostrar u ocultar la contraseña, y el sistema debe validar el PIN antes de pasar a la pantalla de menú.

#1  IMPLEMENTACIÓN — Cambiar ícono del botón ojo (EYE_OPEN / EYE_CLOSE)	
Contexto	Botón de mostrar/ocultar contraseña en login_SCREEN
Problema	El usuario necesitaba alternar entre ícono de ojo abierto (\uF06E) y cerrado (\uF070) al presionar el botón, y además cambiar el modo contraseña del TextArea.
Solución aplicada	1. Crear variable global 'eyeOpen' de tipo boolean, valor inicial true. 2. En el Flow del evento CLICKED del botón ojo: - Nodo IsTrue evaluando 'eyeOpen' - Rama Yes: LVGL Label Set Text → \uF070, SetVariable eyeOpen ← false

	- Rama No: LVGL Label Set Text → \uF06E, SetVariable eyeOpen ← true 3. Para el TextArea: se usaron DOS TextAreas superpuestos en la misma posición. - 'textarea_password': Password mode activado, Hidden = !eyeOpen - 'textarea_plain': Password mode desactivado, Hidden = eyeOpen - Ambos apuntan a la misma variable 'output_keyboard'
Aprendizaje clave	EEZ Studio no tiene acción LVGL para cambiar Password mode en tiempo de ejecución. El campo Password mode del TextArea es un checkbox estático sin Expression binding. La solución correcta es duplicar el TextArea con distintos modos y alternar su visibilidad. El flag Hidden sí acepta Expression, lo que permite reactividad automática.
Notas adicionales	El SwitchCase sobre el texto del label fue el enfoque inicial pero no es confiable. Usar una variable booleana es siempre más robusto.

#2 ● ERROR DEL ASISTENTE — Error del asistente: Expression binding en Password mode

Contexto	Sugerencia inicial para cambiar el modo contraseña del TextArea
Problema	El asistente sugirió buscar un ícono f(x) o = junto al checkbox de Password mode para vincular una expression. Este ícono NO existe porque Password mode es un campo estático sin expression binding.
Solución aplicada	Se investigó la documentación oficial y el repositorio de EEZ Studio. Se confirmó que Password mode no tiene expression binding. La solución correcta fue usar dos TextAreas superpuestos (entrada #1).
Aprendizaje clave	No todos los campos de propiedades en EEZ Studio aceptan expressions. Solo campos de tipo texto/número tienen el selector Literal/Expression. Los checkboxes booleanos como Password mode son siempre estáticos.

#3 ● IMPLEMENTACIÓN — Lógica de escritura del teclado numérico

Contexto	ButtonMatrix como teclado numérico para el login
Problema	Hacer que cada botón numérico agregue su dígito a la variable output_keyboard, con límite de 4 caracteres, y que DEL borre el último.
Solución aplicada	Variable global 'output_keyboard' tipo string, valor inicial "". Variable global 'KEYS' tipo array:string con los valores de cada botón. En el evento del ButtonMatrix: - Evaluate: key = KEYS[Event.getKey(e)] - IsTrue: String.length(key) == 4 (filtra DEL y OK)

	- IsTrue: <code>String.length(output_keyboard) < 4</code> - SetVariable: <code>output_keyboard ← output_keyboard + key</code> Para DEL: <code>String.slice(output_keyboard, 0, String.length(output_keyboard) - 1)</code>
Aprendizaje clave	<code>Event.getKey(e)</code> devuelve el índice del botón presionado en la <code>ButtonMatrix</code>. El array <code>KEYS</code> mapea cada índice a su carácter correspondiente.

3. Indicador de Batería (Header global)

3.1 Objetivo

Mostrar el nivel de batería usando 6 labels con íconos de Font Awesome que representan los estados: vacío, un cuarto, medio, tres cuartos, lleno y cargando. Solo uno debe estar visible en cada momento.

#4  IMPLEMENTACIÓN — Sistema de visibilidad de íconos de batería	
Contexto	Header presente en todas las pantallas
Problema	Primer enfoque: array:boolean con 6 posiciones para controlar visibilidad. Problema: dos posiciones podían quedar en true simultáneamente causando dos íconos visibles. Además, el estado de carga necesitaba mostrar DOS íconos a la vez: el ícono vacío + el rayo.
Solución aplicada	Variable global 'battery_state' tipo integer, valor inicial 0. Convención de valores: 0=vacío, 1=cuarto, 2=medio, 3=tres cuartos, 4=lleno, 5=cargando. Expression en FLAGS → Hidden de cada label: <pre> charge_zero: battery_state != 0 && battery_state != 5 charge_quarter: battery_state != 1 charge_half: battery_state != 2 charge_threeQuarter: battery_state != 3 charge_full: battery_state != 4 charge_on (rayo): battery_state != 5 </pre> Para el estado 5 (cargando) se ven <code>charge_zero</code> Y <code>charge_on</code> simultáneamente.
Aprendizaje clave	Una variable integer es siempre más segura que un array de booleanos para estados mutuamente excluyentes. El flag Hidden con expression es reactivo: se actualiza automáticamente al cambiar la variable.

Se pueden mostrar múltiples widgets simultáneamente ajustando la expresión de cada uno.

4. Pantalla de Menú Principal (MENU)

4.1 Objetivo

Pantalla de tratamiento con selección de LEDs por longitud de onda (R, V, IR), timer de tiempo ajustable, visualización de dosis puntual y dosis total, y botones de Play, Stop y Reset.

#5 ● IMPLEMENTACIÓN — Diseño visual con Containers de fondo azul claro	
Contexto	Crear bloques visuales como panel de TIEMPO y panel de DOSIS
Problema	Los Containers en LVGL tienen fondo transparente por defecto. Al cambiar el color de fondo en STYLE no se veía nada.
Solución aplicada	En STYLE → Main part → DEFAULT → BACKGROUND: 1. Activar el checkbox 'Opacity' y poner valor 255. 2. Activar el checkbox 'Color' y seleccionar el color deseado. Sin el Opacity en 255, el color de fondo es invisible sin importar qué color se elija. Para esquinas redondeadas: STYLE → POSITION AND SIZE → Radius → valor entre 10 y 20.
Aprendizaje clave	En LVGL, background opacity y background color son propiedades independientes. La opacity debe estar explícitamente en 255 (LV_OPA_COVER) para que el color sea visible.

#6 ● IMPLEMENTACIÓN — Checkboxes de LEDs vinculados a variables y color de LEDs	
Contexto	Checkboxes R, V, IR para activar longitudes de onda
Problema	Vincular cada checkbox a una variable booleana y hacer que los LEDs visuales (widget Led) cambien de color según el estado del checkbox.
Solución aplicada	Variables: check_R, check_V, check_IR de tipo boolean, valor inicial false. En STATES → Checked de cada Checkbox: cambiar a Assignable y escribir la variable correspondiente. Para los LEDs: en SPECIFIC → Color → cambiar dropdown a Expression:

	LEDs rojos (4): <code>check_R ? 0xff0000 : 0x000000</code> LEDs violetas (3): <code>check_V ? 0x8800ff : 0x000000</code> LEDs IR (3): <code>check_IR ? 0x8b0000 : 0x000000</code> Nota: los colores se expresan como enteros hexadecimales (0x...), no como strings (#...).
Aprendizaje clave	El campo Checked de un Checkbox acepta modo Assignable para binding bidireccional con variables. El campo Color en SPECIFIC del widget LED sí acepta Expression y usa <code>lv_led_set_color()</code> directamente. Los colores en expressions de EEZ Flow van con prefijo 0x, no con #.

#7 ● ERROR DEL ASISTENTE — Error del asistente: Color rojo del LED en STYLE

Contexto	Intento de configurar el color de los LEDs mediante STYLE en lugar de SPECIFIC
Problema	El asistente sugirió configurar el color mediante STYLE → CHECKED → Background Color. Al probar, el color rojo (#ff0000) aparecía como marrón oscuro, no como rojo brillante. El color violeta sí se veía correctamente.
Solución aplicada	Investigación en el foro oficial de LVGL: el widget Led aplica un factor de brillo mínimo (<code>LV_LED_BRIGHT_MIN = 80/255 ≈ 31%</code>) al color en estado DEFAULT. Esto convierte #ff0000 en aprox. #500000 (marrón oscuro). El violeta resiste mejor este oscurecimiento porque mezcla canales azul y rojo. La solución correcta: usar SPECIFIC → Color → Expression (entrada #6). El sistema de STYLE y el sistema de SPECIFIC son independientes. <code>lv_led_set_color()</code> no pasa por el sistema de brillo de la misma manera.
Aprendizaje clave	El widget Led tiene su propio sistema de brillo interno que oscurece los colores configurados via STYLE. La única forma confiable de controlar el color de un Led es mediante el campo Color en SPECIFIC con Expression.

#8 ● ERROR DEL ASISTENTE — Timer con formato MM:SS — Error IntegerToString

Contexto	Mostrar el tiempo de tratamiento en formato minutos:segundos
Problema	El asistente sugirió usar la función <code>IntegerToString()</code> que NO existe en EEZ Flow. Luego sugirió la expresión: <code>(treatment_time / 60) + ":" + ...</code> que producía decimales: '1.33333333:20' en lugar de '1:20'.

Solución aplicada	La división en EEZ Flow siempre es flotante. Para obtener la parte entera se usa <code>Math.floor()</code>. Expression correcta para el Text del label de tiempo: <code>Math.floor(treatment_time / 60) + ":" + (treatment_time % 60 < 10 ? "0" : "") + (treatment_time % 60)</code> El operador <code>+</code> con un string como <code>":"</code> fuerza la conversión automática del resto a string.
Aprendizaje clave	EEZ Flow NO tiene <code>IntegerToString()</code> ni <code>FloatToString()</code> confiables. La conversión a string se logra concatenando con un string vacío o con otro string. <code>Math.floor()</code> está disponible y es la forma correcta de truncar a entero. El operador <code>%</code> (módulo) funciona correctamente en EEZ Flow para obtener segundos restantes.

#9 ● IMPLEMENTACIÓN — Botones + y - con pulsación continua (LONG_PRESSED_REPEAT)

Contexto	Ajuste de tiempo de tratamiento paso de 5 segundos
Problema	Con el evento <code>PRESSED</code> el tiempo subía demasiado rápido. Se necesitaba un toque que sume 5, y mantener presionado que sume de forma más lenta.
Solución aplicada	Dos event handlers por botón: <code>CLICKED</code>: suma/resta 5 segundos inmediatamente. <code>LONG_PRESSED_REPEAT</code>: suma/resta 5 con un Delay de 300ms antes. <code>LONG_PRESSED_REPEAT</code> se dispara repetidamente mientras se mantiene el botón. El Delay controla la velocidad: 200ms=rápido, 300-500ms=cómodo. Validación en ambos: <code>IsTrue treatment_time > 0</code> antes de restar, <code>IsTrue < 3600</code> antes de sumar.
Aprendizaje clave	<code>LV_EVENT_LONG_PRESSED_REPEAT</code> es el evento nativo de LVGL para repetición al mantener presionado. Combinado con un Delay antes de la acción, permite controlar la velocidad de repetición sin tocar código C.

#10 ● ERROR DEL ASISTENTE — Fórmulas de Dosis Puntual y Dosis Total — Error `FloatToString`

Contexto	Labels que muestran dosis en Joules con 2 decimales
Problema	El asistente sugirió usar <code>FloatToString()</code> que no existe en EEZ Flow. <code>String.Format()</code> tiene un bug conocido en EEZ Studio (issue #643): en modo Run muestra el especificador en lugar del valor.
Solución aplicada	Variables auxiliares: <code>dosis_p_c</code> y <code>dosis_t_c</code> de tipo integer (valores $\times 100$). <code>SetVariable dosis_p_c ← Math.round(0.04 * treatment_time * 100)</code>

	SetVariable dosis_t_c ← Math.round(((check_R ? 4 : 0) * 0.04 + (check_V ? 3 : 0) * 0.04 + (check_IR ? 3 : 0) * 0.04) * treatment_time * 100) Expression en label DOSIS PUNTUAL: Math.floor(dosis_p_c / 100) + "." + (dosis_p_c % 100 < 10 ? "0" : "") + dosis_p_c % 100 Expression en label DOSIS TOTAL: igual con dosis_t_c Potencia: 40mW por LED, 4 rojos + 3 violetas + 3 IR = máx 10 LEDs = 0.4W total
Aprendizaje clave	Para mostrar flotantes con decimales en EEZ Flow: multiplicar por 100, trabajar con enteros, y formatear manualmente la parte decimal. Math.round() evita errores de redondeo de punto flotante. Las variables dosis_puntual y dosis_total de tipo float sugeridas inicialmente fueron innecesarias y se eliminaron.

#11 ● IMPLEMENTACIÓN — Sistema Play/Stop/Reset con User Action countdown_tick

Contexto	Control del timer de tratamiento
Problema	Implementar un contador regresivo con comportamiento lógico completo: Play inicia, Stop pausa, Reset limpia. Los botones deben deshabilitarse según el estado para evitar comportamientos no esperados.
Solución aplicada	Variable 'is_running' boolean, valor inicial false. FLAGS → DISABLED con Expression en cada botón: PLAY: is_running treatment_time == 0 STOP: !is_running RESET: is_running + y -: is_running User Action 'countdown_tick' con loop via Label IN/OUT: Label IN: tick → IsTrue is_running && treatment_time > 0 No: SetVariable is_running ← false → End Yes: Delay 1000 → SetVariable treatment_time ← treatment_time - 1 → IsTrue treatment_time == 0 Yes: SetVariable is_running ← false → End No: Label OUT: tick (regresa al inicio) PLAY: SetVariable is_running ← true → Call User Action countdown_tick STOP: SetVariable is_running ← false (el loop detecta el cambio y termina solo) RESET: SetVariable treatment_time ← 0, check_R/V/IR ← false
Aprendizaje clave	El mecanismo Label IN / Label OUT es la forma nativa de crear loops en EEZ Flow. El loop detecta el cambio de is_running en cada iteración (cada 1 segundo máximo). Los flags DISABLED con Expression evitan comportamientos no esperados sin ninguna lógica adicional.

Call User Action es asíncrono: no bloquea la UI mientras el countdown corre.

5. Pantalla Cambio de Contraseña (CAMBIO_PASSWORD)

#12 ● IMPLEMENTACIÓN — Teclado contextual: aparece al tocar TextArea, desaparece al tocar fuera

Contexto	Teclado numérico compartido entre tres campos de contraseña
Problema	La ButtonMatrix es grande y no puede estar siempre visible. Debe aparecer solo al tocar un TextArea específico y desaparecer al tocar fuera.
Solución aplicada	Variable 'keyboard_visible' boolean, valor inicial false. Variable 'active_field' integer, valor inicial 0. ButtonMatrix: FLAGS → Hidden → Expression: !keyboard_visible Cada TextArea tiene dos eventos: FOCUSED: SetVariable active_field ← N, SetVariable keyboard_visible ← true DEFOCUSED: SetVariable keyboard_visible ← false La ButtonMatrix escribe en la variable del campo activo vía SwitchCase sobre active_field: caso 1: pass_current ← pass_current + key caso 2: pass_new ← pass_new + key caso 3: pass_confirm ← pass_confirm + key
Aprendizaje clave	Los eventos FOCUSED y DEFOCUSED del TextArea son el mecanismo nativo de LVGL para detectar cuándo el usuario interactúa con un campo. Hidden con Expression hace el show/hide completamente automático.

#13 ● ERROR DEL USUARIO — Variable stored_password visible en TextArea

Contexto	Pantalla de cambio de contraseña
Problema	La variable pass_current tenía valor inicial '1234' porque también se usaba para la validación del login. Al entrar a la pantalla de cambio de contraseña, el TextArea mostraba '1234' directamente, revelando la contraseña al usuario.
Solución aplicada	Separar responsabilidades con dos variables: stored_password (string, valor '1234'): contraseña real almacenada. NUNCA vinculada a ningún TextArea.

	pass_current (string, valor "): lo que el usuario escribe en el campo de contraseña actual. En el login: comparar output_keyboard == stored_password (no pass_current). En CAMBIO_PASSWORD al guardar: 1. IsTrue pass_current == stored_password → si no: notificación error 2. IsTrue pass_new == pass_confirm → si no: notificación error 3. IsTrue String.length(pass_new) == 4 → si no: notificación 'Se necesitan 4 dígitos' 4. SetVariable stored_password ← pass_new 5. Limpiar pass_current, pass_new, pass_confirm ← "
Aprendizaje clave	Una variable no debe tener dos responsabilidades simultáneamente. La fuente de verdad (stored_password) nunca debe vincularse a un widget visible. Los campos de entrada siempre deben iniciar vacíos (") y la comparación se hace contra la variable interna.

#14 ● IMPLEMENTACIÓN — Persistencia de variables entre sesiones debug

Contexto	Comportamiento de stored_password al reiniciar debug
Problema	Al salir de debug y volver a entrar, stored_password siempre regresaba a '1234' aunque se hubiera cambiado. El usuario preguntó si era bug visual o problema real.
Solución aplicada	Es comportamiento esperado y completamente normal. Las variables de EEZ Flow viven en RAM. Cada vez que se reinicia debug o se enciende el hardware, la RAM se inicializa y las variables toman sus valores por defecto. Para persistencia real en hardware ESP32 se necesita NVS (Non-Volatile Storage) implementado en código C nativo, llamado desde EEZ Flow via User Actions.
Aprendizaje clave	EEZ Flow no tiene mecanismo automático de persistencia. Todo lo que no se escribe en memoria no volátil (NVS, EEPROM, etc.) se pierde al apagar el dispositivo. Esto es por diseño del sistema, no un bug.

6. Widgets, Estilos y Soluciones Visuales

#15 ● IMPLEMENTACIÓN — Tamaño del Checkbox y texto encima del indicador

Contexto	Checkboxes R, V, IR con etiqueta arriba
-----------------	---

Problema	EEZ Studio no permite redimensionar el cuadro indicador del Checkbox directamente. LVGL no soporta posicionar el texto del Checkbox encima del indicador, solo a la derecha.
Solución aplicada	Para tamaño: en STYLE asignar un font de mayor tamaño. El cuadro crece con la altura del font. Para texto encima: construir el Checkbox manualmente con piezas separadas dentro de un Container: - Un Label con la letra (R, V, IR) en la parte superior del Container. - Un Checkbox sin texto (campo de texto vacío) debajo del Label. - Los eventos del Checkbox funcionan normalmente, solo cambia el layout visual.
Aprendizaje clave	El widget Checkbox de LVGL tiene limitaciones de layout. La solución estándar para layouts personalizados es construir el widget a partir de componentes básicos dentro de un Container.

#16 ● IMPLEMENTACIÓN — Sistema de notificaciones con auto-desaparición y colores por tipo

Contexto	Mensajes informativos para el usuario (error, éxito, advertencia)
Problema	El widget MessageBox de EEZ Studio está marcado como 'work in progress' y tiene funcionalidad muy limitada. Se necesitaban mensajes que aparecieran automáticamente y desaparecieran solos. Se necesitaban colores distintos para error (rojo) y éxito (verde).
Solución aplicada	Variables: show_notification (boolean), notification_text (string). Container de notificación con Text Label vinculado a notification_text. FLAGS → Hidden → Expression: !show_notification Para cambiar color según tipo: STYLE → DEFAULT → Background = rojo, STYLE → CHECKED → Background = verde. En el Flow según el tipo de mensaje: Error: LVGL Widget-States Remove State CHECKED (pone rojo) Éxito: LVGL Widget-States Add State CHECKED (pone verde) Luego: SetVariable notification_text ← 'mensaje' SetVariable show_notification ← true Delay 1000-2000ms SetVariable show_notification ← false Para centrar el texto: en el Label, Width = 100% del padre, STYLE → TEXT → Text align = CENTER.
Aprendizaje clave	No se puede cambiar el background color via Expression directamente (feature request pendiente issue #450). La alternativa es usar el sistema de States (CHECKED/DEFAULT) con estilos por estado.

LVGL Widget-States Add/Remove State es la acción correcta para cambiar estados desde Flow.
El Label con texto variable necesita ancho fijo (100%) para que el centrado funcione correctamente.

#17 ● IMPLEMENTACIÓN — Agregar imagen (ícono) dentro de un Button

Contexto	Botones con íconos de imagen en lugar de texto (LVGL 9 no tiene todos los íconos en fuente)
Problema	LVGL 9.x no incluye ciertos íconos en sus fuentes de símbolos. Se necesitaba usar una imagen PNG como ícono dentro de un botón.
Solución aplicada	1. Preparar el PNG con fondo transparente (canal alpha) con las dimensiones deseadas. 2. En EEZ Studio: pestaña Bitmaps → botón + → importar el PNG → asignar nombre. 3. Crear el Button y dejar su campo de texto vacío en SPECIFIC. 4. En el Widgets Structure, arrastrar un widget Image como hijo del Button. 5. En SPECIFIC del Image widget: campo Bitmap → seleccionar el bitmap importado. 6. Usar los íconos de alineación de EEZ Studio para centrar la imagen dentro del botón. Los eventos CLICKED del Button siguen funcionando normalmente.
Aprendizaje clave	Los widgets LVGL pueden tener hijos. Un Image hijo de un Button es el patrón correcto para botones con íconos de imagen. El PNG debe tener fondo transparente o el fondo blanco será visible sobre el botón. Si el PNG no es un PNG real (SVG o WebP renombrado), EEZ Studio dará error 'PNG file has invalid signature'.

7. Compilación del Proyecto y Herramientas Externas

#18 ● PROBLEMA TÉCNICO — Error de compilación: pypng, lz4 y pngquant faltantes

Contexto	Primera compilación del proyecto con imágenes bitmap
Problema	Al compilar: 'An error occurred while building images for LVGL 9.x'. EEZ Studio usa el script LVGLImage.py del equipo LVGL para convertir imágenes PNG a arrays C. Este script requiere Python 3, pypng, lz4 y pngquant.

Solución aplicada	1. Instalar pypng: <code>python -m pip install pypng==0.20220715.0</code> 2. Instalar lz4: <code>python -m pip install lz4</code> 3. Descargar pngquant desde https://pngquant.org 4. Agregar la carpeta de pngquant.exe al PATH de Windows. 5. CRÍTICO: Cerrar y volver a abrir EEZ Studio después de modificar el PATH. Las variables de entorno solo se aplican a procesos iniciados después del cambio. Verificación: abrir una terminal NUEVA y ejecutar <code>'pngquant --version'</code> → debe responder <code>'2.17.0'</code>
Aprendizaje clave	EEZ Studio no recarga las variables de entorno de Windows en caliente. Siempre cerrar y reabrir la aplicación después de modificar el PATH. La verificación con una terminal nueva (no la misma donde se hizo el cambio) confirma que el PATH está correcto a nivel de sistema.

#19 ● ERROR DEL USUARIO — Error PNG con firma inválida (archivo no era PNG real)

Contexto	Bitmap de ícono global que no compilaba
Problema	Error: <code>'png.FormatError: PNG file has invalid signature'</code> . El archivo importado como bitmap era un SVG o WebP renombrado con extensión <code>.png</code> , no un PNG real.
Solución aplicada	1. Eliminar el bitmap problemático de la sección Bitmaps de EEZ Studio. 2. Abrir el archivo original en cualquier editor (Paint, GIMP, navegador). 3. Exportar/guardar explícitamente como PNG. 4. Re-importar el archivo PNG correcto en Bitmaps. La compilación funcionó correctamente tras este proceso.
Aprendizaje clave	Cambiar la extensión de un archivo no convierte el formato. EEZ Studio valida la firma interna del archivo PNG (los primeros 8 bytes). Un archivo con extensión <code>.png</code> pero contenido SVG/WebP fallará siempre en la compilación.

#20 ● ERROR DEL USUARIO — Warnings de conexión: `null→string` y `boolean→string` en Flow

Contexto	Pantalla CAMBIO_PASSWORD, validaciones de contraseña
Problema	Warning: <code>'Connection line incompatible data types: null → string'</code> Warning: <code>'Connection line incompatible data types: boolean → string'</code> El proyecto compilaba pero con 5 checks de advertencia.
Solución aplicada	Los nodos en EEZ Flow tienen dos tipos de conexiones: - Entrada de ejecución (secuencia): punto izquierdo sin etiqueta. No lleva datos.

	- Entradas de datos: puntos etiquetados (key, value, etc.) que esperan un tipo específico. El error null→string: se conectaba la salida de ejecución de un nodo LVGL (que no emite valor) a la entrada de datos 'key' de un SetVariable. El error boolean→string: se conectaba la salida 'Yes' de IsTrue (boolean) a la entrada 'key'. Solución: los valores de 'key' y 'value' en SetVariable se escriben directamente como texto en los campos del nodo, NO se conectan con cables desde otros nodos. Los cables solo transportan señal de ejecución (secuencia), no datos.
Aprendizaje clave	En EEZ Flow los datos y la secuencia de ejecución son conceptos separados. Los cables entre nodos = secuencia. Los valores en campos de texto de los nodos = datos. No mezclar los dos tipos de conexión.

8. Referencia Rápida de Expresiones EEZ Flow

Expresiones validadas y funcionando

Propósito	Expresión
Tiempo MM:SS	<code>Math.floor(treatment_time / 60) + ":" + (treatment_time % 60 < 10 ? "0" : "") + (treatment_time % 60)</code>
Dosis puntual (2 dec.)	<code>Math.floor(dosis_p_c / 100) + "." + (dosis_p_c % 100 < 10 ? "0" : "") + dosis_p_c % 100</code>
Dosis total (2 dec.)	<code>Math.floor(dosis_t_c / 100) + "." + (dosis_t_c % 100 < 10 ? "0" : "") + dosis_t_c % 100</code>
LED rojo encendido	<code>check_R ? 0xff0000 : 0x000000</code>
LED violeta encendido	<code>check_V ? 0x8800ff : 0x000000</code>
LED IR encendido	<code>check_IR ? 0x8b0000 : 0x000000</code>
Ocultar batería vacía	<code>battery_state != 0 && battery_state != 5</code>
Ocultar batería cargando	<code>battery_state != 5</code>
Deshabilitar PLAY	<code>is_running treatment_time == 0</code>
Deshabilitar STOP	<code>!is_running</code>
Ocultar teclado	<code>!keyboard_visible</code>
TextArea plain visible	<code>eyeOpen</code>

TextArea password visible	!eyeOpen
---------------------------	----------

9. Inventario de Variables Globales

Variable	Tipo	Default	Propósito
output_keyboard	string	""	Texto escrito en el teclado del login
eyeOpen	boolean	true	Estado del ícono de ojo (mostrar/ocultar PIN)
stored_password	string	"1234"	Contraseña real almacenada (interna, no visible)
pass_current	string	""	Campo: contraseña actual (entrada del usuario)
pass_new	string	""	Campo: nueva contraseña
pass_confirm	string	""	Campo: repetir nueva contraseña
battery_state	integer	0	Estado de batería: 0=vacío,...,4=lleno,5=cargando
check_R	boolean	false	Checkbox LED Rojo seleccionado
check_V	boolean	false	Checkbox LED Violeta seleccionado
check_IR	boolean	false	Checkbox LED Infrarrojo seleccionado
treatment_time	integer	0	Tiempo de tratamiento en segundos
is_running	boolean	false	Estado del countdown (corriendo o no)
dosis_p_c	integer	0	Dosis puntual × 100 (para mostrar 2 decimales)
dosis_t_c	integer	0	Dosis total × 100 (para mostrar 2 decimales)
keyboard_visible	boolean	false	Controla visibilidad del teclado en CAMBIO_PASSWORD
active_field	integer	0	Campo activo: 1=current, 2=new, 3=confirm
show_notification	boolean	false	Controla visibilidad del container de notificación

<code>notification_text</code>	string	""	Texto del mensaje de notificación al usuario
<code>KEYS</code>	array:string	[...]	Mapa de índice de botón → carácter del teclado
<code>loading_value</code>	integer	100	Valor de la barra de progreso en SplashScreen

10. Lecciones Aprendidas y Limitaciones Conocidas de EEZ Studio

Limitaciones confirmadas de EEZ Studio + LVGL

- Password mode del TextArea: no tiene Expression binding. Solución: dos TextAreas superpuestos.
- Background color: no tiene Expression binding. Solución: sistema de States (CHECKED/DEFAULT).
- FloatToString / IntegerToString: no existen. Solución: concatenación con string y Math.floor().
- String.Format(): tiene bug en modo Run (issue #643). No usar en producción.
- MessageBox: marcado como 'work in progress'. Solución: Container con auto-hide via Delay.
- Color de LED via STYLE: afectado por sistema de brillo interno. Solución: usar SPECIFIC → Expression.
- Texto encima del Checkbox: no soportado por LVGL. Solución: Container con Label + Checkbox separados.
- Persistencia de variables: no existe. Para hardware real se necesita NVS en código C nativo.

Buenas prácticas consolidadas

- Siempre usar una variable entera para estados mutuamente excluyentes (battery_state, active_field).
- Separar responsabilidades: una variable = una responsabilidad (stored_password ≠ pass_current).
- Los flags Hidden, Disabled y States son la forma más eficiente de controlar el comportamiento visual.
- Los cables en EEZ Flow transportan secuencia, no datos. Los datos van en campos de texto de los nodos.
- Math.floor() para división entera. El operador / siempre es división flotante.
- Math.round(valor * 100) para trabajar con 2 decimales usando solo enteros.

- Reiniciar EEZ Studio siempre después de modificar variables de entorno de Windows (PATH).
- Los PNG deben ser PNG reales. Verificar con un editor antes de importar como Bitmap.

11. Estado Actual del Proyecto

Al momento de escribir esta bitácora, las siguientes funcionalidades están implementadas y verificadas en modo Run/Debug de EEZ Studio:

- ☒ SplashScreen con animación de carga
- ☒ Login con PIN de 4 dígitos, teclado numérico y botón mostrar/ocultar
- ☒ Indicador de batería con 6 estados visuales incluyendo modo carga
- ☒ Selección de LEDs con cambio de color reactivo
- ☒ Timer con formato MM:SS, botones +/- con pulsación continua
- ☒ Cálculo automático de Dosis Puntual y Dosis Total con 2 decimales
- ☒ Sistema Play/Stop/Reset con deshabilitar botones por estado
- ☒ Pantalla de cambio de contraseña con teclado contextual
- ☒ Sistema de notificaciones con auto-desaparición y colores por tipo
- ☒ Compilación del proyecto sin errores (1 warning menor resuelto)

- ☐ Pendiente: integración con hardware real (lectura de batería, control de LEDs físicos)
- ☐ Pendiente: persistencia de contraseña en NVS del hardware
- ☐ Pendiente: comunicación UART/I2C con microcontrolador principal

Esta bitácora debe actualizarse conforme avance el desarrollo de la integración con hardware.